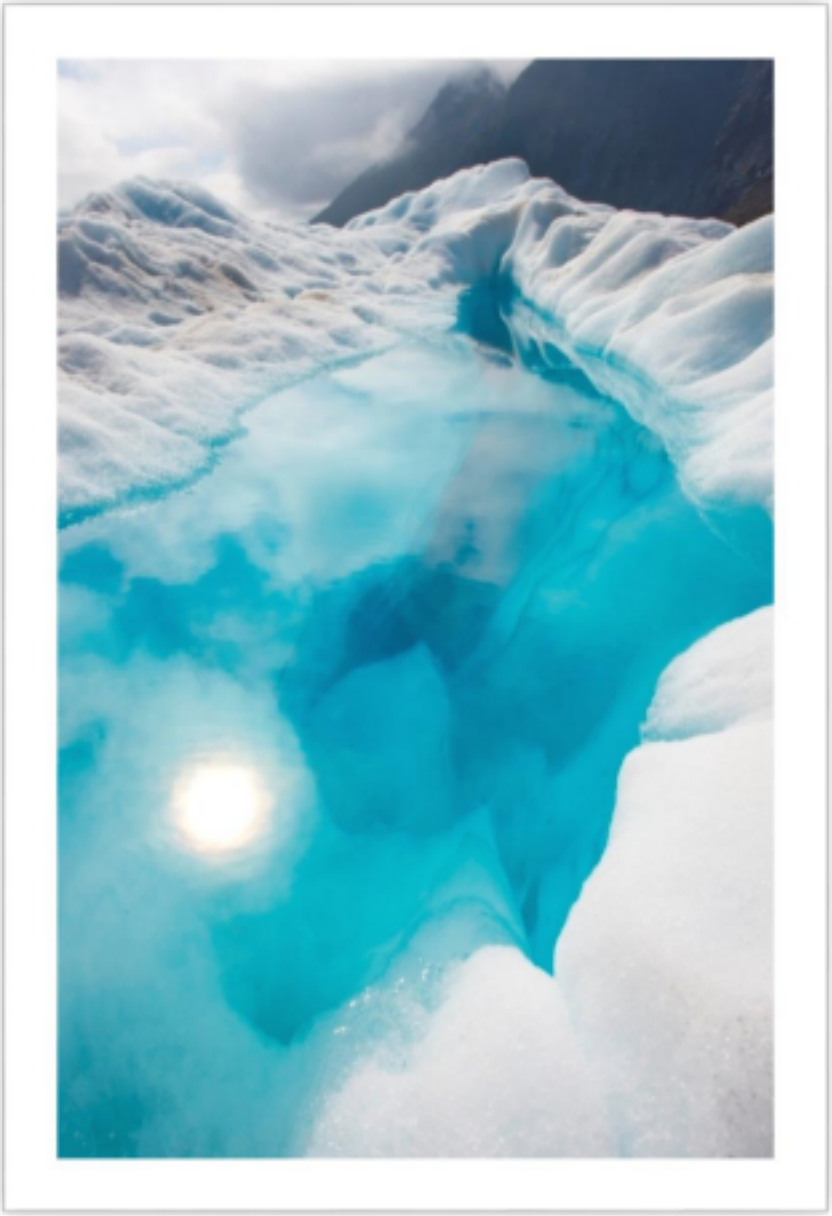


Kravspecifikation

Emil.Oliver,Carl-Emil,Aksel| Programmør | 13. Marts 2026

Indholdsfortegnelse

Kravspecifikation
Customer Communication Platform – GRUPPEBASERET KOMMUNIKATION MED CHATBOT OG NOTIFIKATIONER **4**
1. Indledning 4
2. Generel beskrivelse 5
3. De specifikke krav 10
3.1 Use Cases 11
Usecase: 01 - Register 12
Usecase: 02 - Login 14
Usecase: 03 - Forgot password 16
Usecase: 04 - Opret sag manuelt af supporter 17
Usecase: 05 - Opret sag manuelt af Customer 18
Usecase: 06 - Automatisk oprettelse af sag via e-mail 20
Usecase 07 - Chatbot håndterer henvendelse 21
Usecase 08 – Eskalering fra chatbot til supporter 22
Usecase 09 – Ændre status på sag 23
4. Eksterne grænseflade-krav 23
5. Krav til programmellets ydelse 25
6. Kvalitetsfaktorer 26
7. Andre krav 27
Fordele ved CI/CD 29
Brugen af CI/CD giver flere fordele i projektet: 29
CI/CD i dette projekt 29
8. Skærmbilleder 33
9. E/R diagram 34
10. Estimeret tidsplan 34

Case Beskrivelse - **Customer Communication Platform**

Case Beskrivelse

Customer Communication Platform er en centraliseret kommunikationsløsning, der har til formål at samle en virksomheds kundedialog på én samlet platform. Systemet skal gøre det muligt for virksomheder at håndtere supporthenvendelser struktureret, effektivt og overskueligt.
Platformen skal kunne modtage kundehenvendelser via flere kanaler. Når en kunde kontakter virksomheden, oprettes der automatisk en sag (ticket) i systemet. Sagen fungerer som et centralt kommunikationspunkt, hvor al dialog mellem virksomhed og kunde registreres og gemmes.

Det skal være muligt at oprette en sag på følgende måder:

- Direkte via applikationen
- Automatisk ved at en kunde sender en e-mail til en dedikeret support adresse
- Automatisk oprettelse igennem chatbot (Hvis nødvendigt)

Ved e-mailhenvendelser skal systemet automatisk:

- Registrere den indgående mail
- Oprette en sag i databasen
- Knytte efterfølgende mails til den eksisterende sag
- Gøre det muligt at kommunikere videre direkte via e-mail

Dette sikrer, at kunden kan fortsætte dialogen uden nødvendigvis at logge ind i systemet.
Derudover skal platformen indeholde en hjælpe chat, som kan embeddes i en virksomheds applikation eller hjemmeside. Chatten skal kunne:

- Besvare generelle spørgsmål via en chatbot-funktion
- Guide brugeren gennem simple problemstillinger
- Eskalere samtalen til en supportmedarbejder ved behov
- Oprette en sag automatisk, hvis henvendelsen kræver videre behandling

Systemet skal dermed fungere som en samlet kommunikations hub, hvor alle kundehenvendelser håndteres ensartet og struktureret.

Funktionerne omfatter:
Platformens funktionalitet omfatter følgende:

- Kommunikation med kunder via e-mail eller direkte gennem applikationen
- Chatbot-udvidelse til besvarelse af generelle spørgsmål samt eskalering til support
- Automatisk oprettelse af sager ved indgående supporthenvendelser

- Intern kommunikation mellem medarbejdere omkring åbne og lukkede sager
- Administration og håndtering af organisationer og deres tilknyttede brugere
- Statusstyring af sager (fx åben, afventer svar, lukket)

Anvendt teknologi

Frontend:

1. HTML og CSS til struktur og styling
   Grunden til valget bag HTML og CSS er fordi at det er grundstenen i al webudvikling og fordi at alle browsere forstår dem direkte, og vi har god erfaring med dem.
2. Razor Pages til server-side rendering og dynamisk visning af data
   Grunden til valget bag Razor pages er fordi den lader og skrive C#-logik og HTML i samme fil og fordi vi har erfaring med det, og det er simpelt og passer godt ind med resten af teknologierne som også bruger .NET

Backend

3. ASP.NET Web API til udvikling af services og API-endpoints
   Grunden til valget af ASP.NET Web API er fordi det er hurtigt og fungerer naturligt med resten af .NET og vi har igen god erfaring med det.
4. Entity Framework som ORM (Object-Relational Mapping) til håndtering af database operationer
   Grunden til valget af Entity Framework er fordi at vi kan arbejde med normale C#-objekter i stedet for rå SQL og fordi vi har erfaring med det.

Database

5. PostgreSQL som primær database
   Grunden til valget af PostgreSQL er fordi den er en gratis og velkendt database med mange udvidelser. Den understøtter bl.a. pgvector, som er nødvendig da vi vil bruge AI-funktioner som semantisk søgning.

Service-arkitektur

6. .NET Aspire til orkestrering og håndtering af services
   Grunden til valget af .NET Aspire er fordi det automatisk starter vores services i den rigtige rækkefølge, kobler dem sammen og giver os et samlet overblik over det hele under udvikling.

Definitioner og forkortelser

Tenant	Vores kunde som har købt adgang til vores service
Customer	Vores Tenants kunder.

FAQ	Frequently Asked Questions

Problemformuleringsspørgsmål:

1. Hvordan kan vi designe og implementere en centraliseret platform, der samler kundekommunikation fra flere kanaler i én sagshåndtering, så at support arbejdet bliver mere overskueligt og effektivt?
2. Hvordan kan systemet automatisk oprette og håndtere sager fra indgående e-mails, og sikre at efterfølgende beskeder knyttes til den korrekte sag for at bevare en stabil tovejskommunikation?
3. Hvordan kan en embedded-hjælp-chat designes til at løse simple henvendelser via chatbot, men samtidig eskalere til en supportmedarbejder med bevaret kontekst og kun oprette en sag når det er nødvendigt?
4. Hvordan kan vi bygge en skalerbar servicearkitektur med rollebaseret adgang, der samtidig gør det muligt at udvide platformen med nye kanaler og funktioner over tid?
5. Hvordan kan vi sikre sporbarhed og ansvarlig håndtering af persondata i systemet, så det er muligt at se hvem der har gjort hvad og hvornår og så platformen lever op til GDPR?

Kravspecifikation

Customer Communication Platform – GRUPPEBASERET KOMMUNIKATION MED CHATBOT OG NOTIFIKATIONER

1. Indledning

1.1 Formål

Formål

En applikation som kan samle virksomheders kommunikation i et centralt system for at gøre det lettere for dem at holde styr på alle åbne support sager og andre kundehenvendelser.

Hvad skal det bruges til

Systemet skal anvendes af virksomheder til at samle og strukturere kundekommunikation (e-mail, chatbot og direkte henvendelser) i et sagshåndteringssystem (ticket system), så support arbejde bliver mere effektivt og overskueligt.

Hvem er udvikleren

Udvikles af projektgruppen (Aksel, Carl, Emil og Oliver).

Navn	Strength	Weakness

Aksel		
Carl		
Emil	- Backend (C#,Database,Api,Unit tests) - Kommunikation - Hurtig læringsevne	- Prioritering under pres - Frontend (HTML,CSS)
Oliver		

Hvem er kunden

Virksomheder med behov for struktureret kundesupport og kommunikation håndtering

1.3 Læsevejledning

Dokumentet beskriver CCP Appens funktionalitet, arkitektur og krav**.** Kravene er opdelt i generelle beskrivelser, funktionelle krav, grænseflader og kvalitetskrav.

2. Generel beskrivelse

2.1 Systembeskrivelse

Systemet bliver designet som en SaaS (Software as a Service) og indeholder derfor ikke noget særligt hardware hos brugeren. Løsningen bliver hostet på server eller i cloud og tilgås gennem en almindelig webbrowser. Den centrale del af systemet er en webapplikation lavet med razor pages, som fungerer som brugerens adgang til platformen og gør det muligt at håndtere kundedialog, sager og beskeder et samlet sted.

Derudover bliver systemet bygget med en serviceorienteret arkitektur så de forskellige dele af systemet kan holdes adskilt og udvides løbende. Det gør det lettere at vedligeholde og skalere løsningen i fremtiden, hvis der kommer flere brugere, flere virksomheder eller behov for nye funktioner. En af de services systemet består af er en e-mail service, som har til formål at modtage indgående support mails, oprette sager automatisk og sende svar eller notifikationer via mail.

Til datalagring bruges PostgreSQL databaser. Da systemet skal håndtere data som brugere, organisationer, sager, beskeder og statusser giver det bedst mening at bruge en relationsdatabase som PostgreSQL. Samtidig er PostgreSQL en meget udbredt og stabil database, og den understøtter også

udvidelser, som kan være relevante i forbindelse med chatbot integrering og AI-funktioner.

Systemet vil også indeholde en chatbot service, som bruger Ollama og en tilhørende AI-model. Denne service skal bruges til at besvare generelle spørgsmål fra kunder, hjælpe med simple problemstillinger og eskalere henvendelser videre til en supportmedarbejder eller oprette en sag, hvis det bliver nødvendigt. På den måde fungerer systemet både som en kommunikationsplatform og som et supportværktøj, der kan automatisere en del af den første kontakt med kunden.

Derudover vil vi benytte os af Keycloak som authentication provider til login, roller og adgangsstyring, så forskellige brugertyper som admin, supporter manager og customer kan få adgang til de relevante funktioner i systemet. Samlet set bliver systemet en central platform som samler kundekommunikation fra flere kanaler og gør den mere overskuelig og struktureret for virksomheder.

Systemet skal bestå af:

- Webapplikation (Razor pages)
- Database (PostgreSQL)
- Backend (C#)
- Keycloak (Authentication provider)
- E-mail service (C#)
- Chatbot service (Ollama/C#)

Hardware

systemet bliver hosted på server/cloud og tilgås via browser

2.2 Programmets funktion

Programmet gør det muligt for virksomheder at samle og håndtere kundekommunikation i et centralt system. Det overordnede formål er, at kundehenvendelser fra flere forskellige kanaler bliver samlet et sted, så virksomheder får et bedre overblik over dialogen og kan håndtere deres support mere struktureret. Systemet skal derfor fungere som et ticket system, hvor sager oprettes, opdateres og afsluttes gennem den daglige kommunikation med kunden.

Programmet gør det muligt at oprette sager direkte i applikationen, automatisk via e-mail og automatisk gennem chat-botten, hvis en henvendelse kræver videre behandling. Ved e-mailhenvendelser skal systemet kunne registrere den indgående mail, oprette en sag og sørge for at efterfølgende mails bliver knyttet til den korrekte eksisterende sag. På den måde kan kunden fortsætte dialogen via e-mail uden nødvendigvis at logge ind i systemet.

Derudover gør systemet det muligt for supportmedarbejdere at se åbne og lukkede sager, følge historikken på en sag, ændre status på sager og kommunikere videre med kunden direkte fra platformen. Systemet skal også understøtte intern kommunikation mellem medarbejdere omkring sager, så flere personer i virksomheden kan samarbejde om at løse en henvendelse.

Programmet indeholder også en embedded hjælpe chat, som kan bruges på virksomhedens hjemmeside. Denne chat gør det muligt for kunder at stille spørgsmål direkte, få hjælp til simple problemstillinger og modtage svar fra en chatbot. Hvis chatbotten ikke kan løse problemet, skal systemet kunne eskalere samtalen til en supportmedarbejder eller oprette en sag automatisk, så henvendelse kan behandles videre i systemet.

Systemet skal kunne køre på:

- Moderne browsere
- 64-bit OS
- Internetforbindelse

Server krav:

- ASP.NET runtime
- PostgreSQL database
- Mailserver
- Ollama / Chatbot

2.3 Programmets begrænsninger

Programmet vil blive udviklet som en webapplikation og kan derfor ikke bruges offline, da det kræver en internetforbindelse for at kunne kommunikere med server, database, e-mail service og chatbot service. Systemet vil derfor være afhængig af at brugeren har adgang til nettet for at kunne oprette, se og håndtere sager.

Systemet er heller ikke designet til at integrere med sociale medier eller andre eksterne besked platforme som for eksempel Facebook Messenger, Instagram eller WhatsApp. Kundekommunikation håndteres i stedet gennem systemets egne funktioner, e-mail og den indbyggede chat løsning.

Derudover er systemet ikke tænkt som en fuld erstatning for virksomhedens eksisterende e-mailsystem. Det fungerer i stedet som en platform, der kan modtage og sende e-mails i forbindelse med kundesager, men det skal ikke bruges som et generelt mailsystem til al intern eller ekstern kommunikation.

Systemet understøtter ikke upload af videoer, kommunikationen vil derfor primært være baseret på tekst og eventuelt enklere vedhæftninger, hvis dette senere bliver understøttet. På samme måde vil chatbotten kun fungere som

tekstbaseret hjælp og vil ikke kunne bruges med tale, stemmestyring eller diktering.

Endelig er systemet afgrænset til kundekommunikation og sagsbehandling, hvilket betyder, at det ikke er udviklet til at håndtere andre typer af forretningsprocesser uden for dette område.

2.4 Programmets fremtid

Systemets forventede levetid er mere end fem år. Arkitekturen designes skalerbar og udvidelsesvenlig med henblik på, at systemet kan videreudvikles og tilpasses nye behov i fremtiden. Det betyder, at løsningen ikke kun er tænkt som et kortsigtet projekt, men som en platform der løbende kan bygges videre på, efterhånden som virksomheder får behov for flere funktioner og mere avanceret kundehåndtering.

I fremtiden vil systemet kunne udvides med integration til flere kommunikationskanaler, så kundedialog fra eksempelvis sociale medier eller andre eksterne beskedplatforme kan samles i systemet. Derudover vil chatbot-funktionen kunne videreudvikles, så den bliver mere intelligent, bedre til at forstå kundens spørgsmål og i højere grad kan hjælpe med automatisering af supportforløb.

Systemet vil også kunne udvides med flere administrative funktioner, mere avanceret statistik og rapportering samt bedre muligheder for at analysere kundekommunikation og svartider. På den måde vil platformen med tiden kunne udvikle sig fra et simpelt ticket og kommunikationssystem til et mere komplet værktøj til kundeservice og supportstyring

Da systemet er bygget med fokus på skalerbarhed, vil det også være muligt at tilpasse det til en større mængde brugere, organisationer og sager i fremtiden. Samlet set er programmets fremtid derfor, at det kan videreudvikles i takt med virksomheders behov og teknologiske muligheder.

2.5 Brugergrupper

Systemer indeholder flere forskellige typer bruger som kan grupperes i følgende roller:

- Admin
- Supporter
- Customer
- Manager

Rollerne bliver konfigureret gennem Keycloak hvor hver Tenant har en organisation hvor deres bruger bliver oprettet. Når en ny Tenant bliver oprettet, bliver der ligeså oprettet en organisation samt en gruppe.

Gruppen hører til den specifikke organisation/tenant, den skal så indeholde fire grupper: Admin, Supporter,Customer og Manager. Hver gruppe skal så have deres dertilhørende rolle. Ved at grupperne får rollerne, gør det lettere at tilføje og fjerne roller fra en given bruger, da man bare skal fjerne eller tilføje brugeren fra gruppen.

Det gør det også lettere at supportere og formindske systemets kompleksitet ved at begrænse antallet af steder hvor brugerroller/rettigheder er defineret.

	Admin	Manager	Support	Customer
Rettigheder til at oprette Tickets	✅	✅	✅	✅
Rettigheder til at tildele tickets til supporter/brugere	✅	✅	❌	❌
Rettigheder til se alle Tenants Tickets	✅	✅	❌	❌
Rettigheder til at se alle Customers Tickets	✅	❌	❌	✅
Rettigheder til at oprette nye customers og til at redigere eksisterende	✅	❌	❌	❌
Rettigheder til at tilføje nye bruger/supporter og redigere brugers roller	✅	❌	❌	❌

2.6 Krav til udviklingsforløbet

Udvikling udføres agilt med version systering gennem git og der bliver løbende udført kode reviews. Derudover udarbejdes der dokumentation for systemarkitektur, database, API og brugerfunktioner. Ændringer i krav eller funktionalitet bliver godkendt gennem gruppen eller den potentielle kunde. Det programmeringssprog der bliver i benyttet er C# da vi er mest teknisk stærke

og erfarende inde for lige præcis dette sprog. Vi er i stand til at udnytte det på bedst mulig vis, og der igennem få det bedste resultat muligt.

2.7 Omfang af kundeleverance

Kunden modtager en færdig webapplikation med tilhørende databaseopsætning, chatbot-funktionalitet og ticket-funktionalitet. Det betyder, at kunden får adgang til en samlet platform, hvor kundehenvendelser kan modtages, registreres og behandles i systemet.

Kundeleverancen omfatter også den nødvendige opsætning af systemets grundlæggende funktioner, herunder brugerhåndtering, sagsoprettelse, beskedhåndtering og adgang til den indbyggede chat og e-mail relaterede funktionalitet. Derudover leveres den tekniske struktur, så systemet kan installeres, hostes og anvendes som en webbaseret løsning.

Udover selve programmet modtager kunden også teknisk dokumentation og brugerrettet dokumentation. Den tekniske dokumentation skal gøre det muligt at forstå systemets opbygning, installation og videre vedligeholdelse, mens den brugerrettede dokumentation skal hjælpe slutbrugere og administratorer med at anvende systemet korrekt i praksis.

Kundeleverancen omfatter dermed både selve softwareløsningen og den nødvendige dokumentation, så systemet kan tages i brug og videreføres efter aflevering.

2.8 Forudsætninger

Kunden skal give information i forhold til deres login/bruger

3. De specifikke krav

3.1 Data

Hver tenant er i stand til at oprette customers som så kan skabe tickets, de kan også opsætte en integration så at ved nye email kan systemet skabe nye tickets.

3.2 Funktionelle krav

Systemet skal gøre det muligt at oprette og administrere tickets, hvor medarbejdere kan tilføjes, og hvor hver ticket har et emne og en permanent kommunikationshistorik. Brugere skal kunne sende og modtage beskeder i en ticket, og beskeder skal vises i kronologisk rækkefølge med registreret afsender og tidspunkt.

Ved nye kundehenvendelser eller beskeder skal systemet udsende push-notifikationer til relevante medarbejdere. Notifikationer skal begrænses

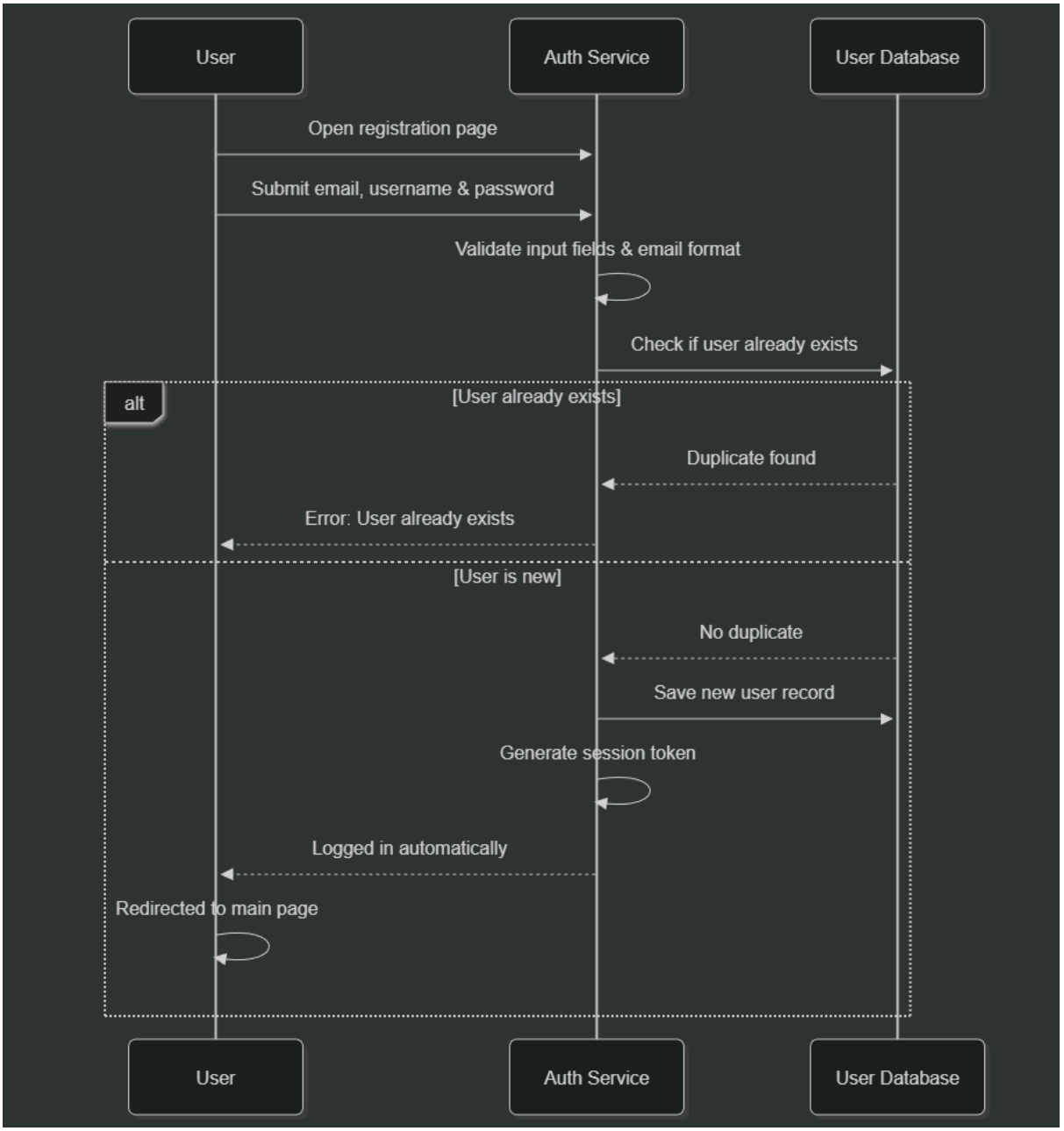
ved høj beskedfrekvens for at undgå unødig forstyrrelse. En medarbejder kan tildeles en ticket af en manager eller administrator, eller de kan selv vælge en ledig ticket. Når en medarbejder tildeles eller vælger en ticket, får de automatisk fulde rettigheder til den pågældende sag.

Chatbotten skal kunne integreres direkte på en tenants website og anvendes til at besvare ofte FAQs samt præsentere tilgængelig information for kunden.

Adgang til tickets og beskeder skal være begrænset til tildelte medarbejdere og den tilknyttede kunde, og login skal være påkrævet for at tilgå platformen. Systemet skal desuden registrere sporbarhed, herunder afsender, tidspunkt og eventuelle ændringer i en sag, så der altid er et fuldt revisionsspor for al kommunikation.

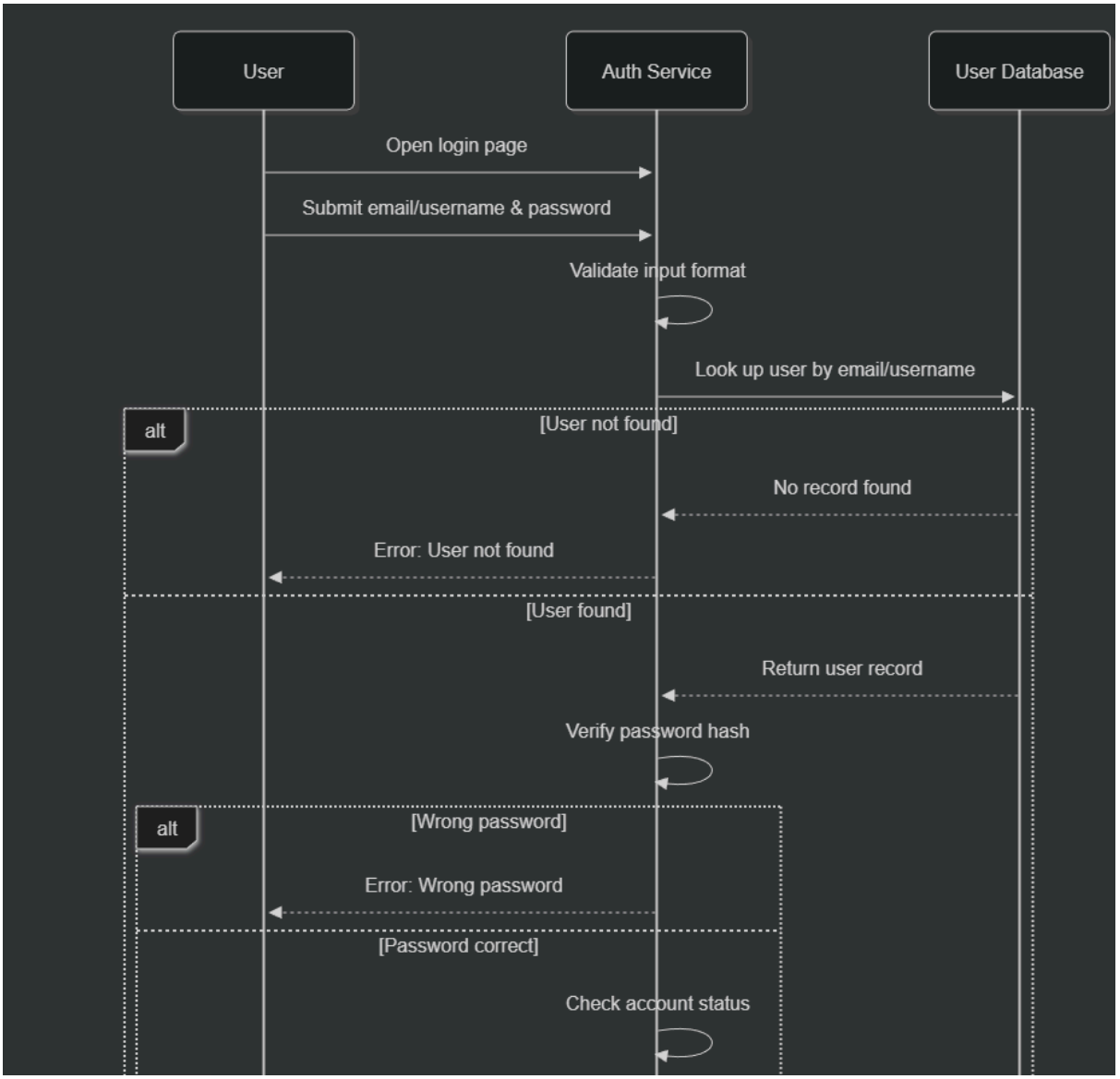
3.1 Use Cases

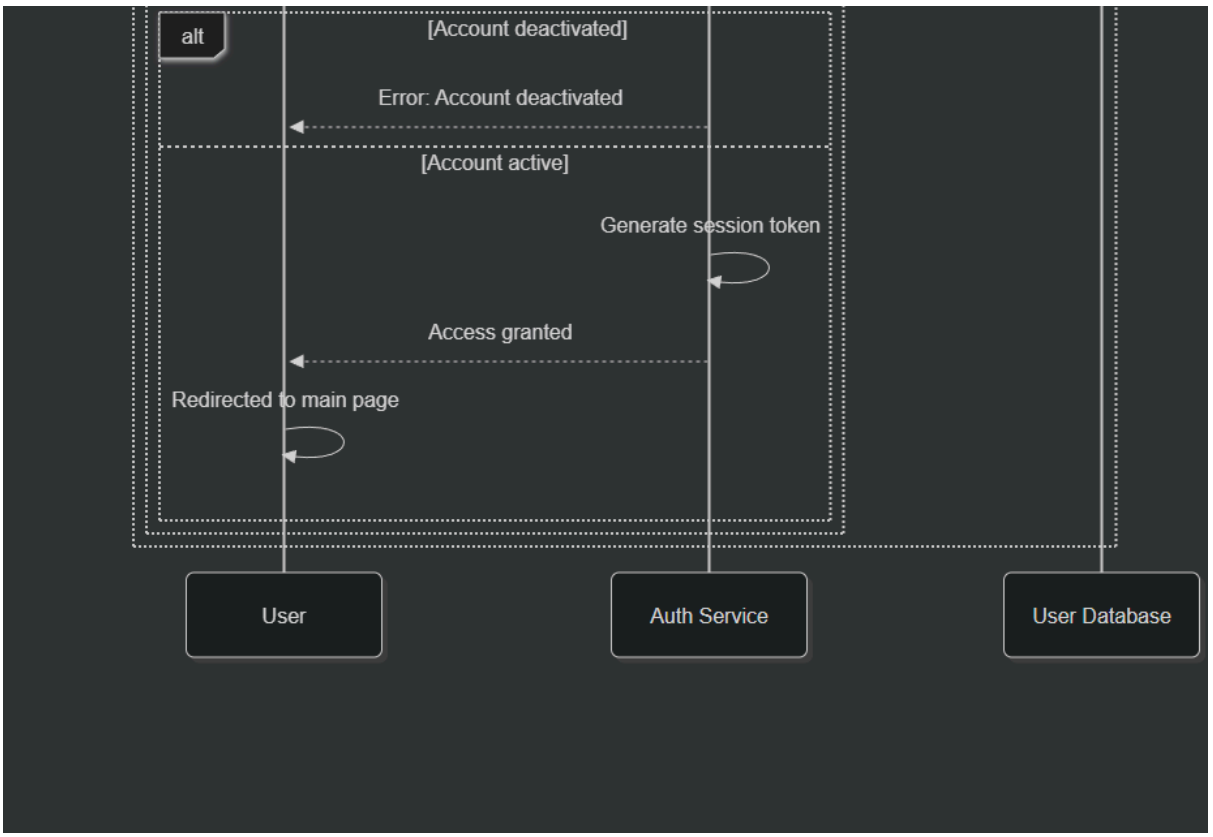
Usecase 01 - Register
Usecase 02 - Login
Usecase 03 - Forgot password
Usecase 04 - Opret sag manuelt - af supporter
Usecase 05 - Opret sag manuelt - af Customer
Usecase 06 - Automatisk oprettelse af sag via e-mail
Usecase 07 - Chatbot håndterer henvendelse
Usecase 08 - Eskalering fra chatbot til supporter
Usecase 09 - Ændre status på sag



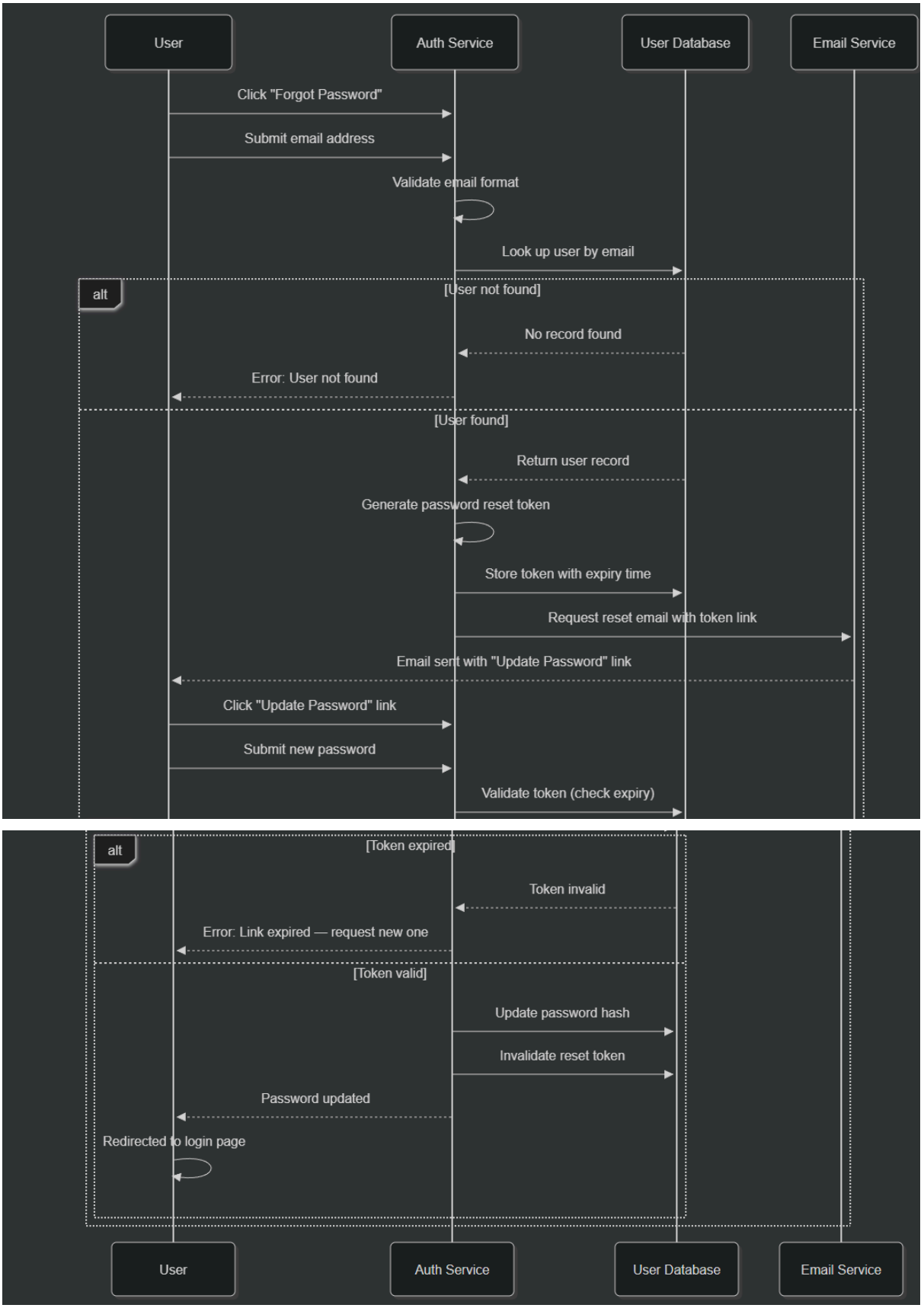
Usecase: 01 - Register
Aktør: Bruger
Forudsætninger: - Brugeren har en e-mail - Systemet er tilgængeligt via en browser - Brugeren har internetforbindelse
Succes scenarie: - Brugeren åbner registrer siden - Brugeren indtaster e-mail, brugernavn og adgangskode

- Brugeren trykker “Register” - Systemet validere oplysningerne - Brugeren logges ind efter en valid registrering
Fejlkilder: - Brugeren indtaster forkert e-mail - Systemet kan ikke forbinde til databasen - Brugeren eksisterer allerede

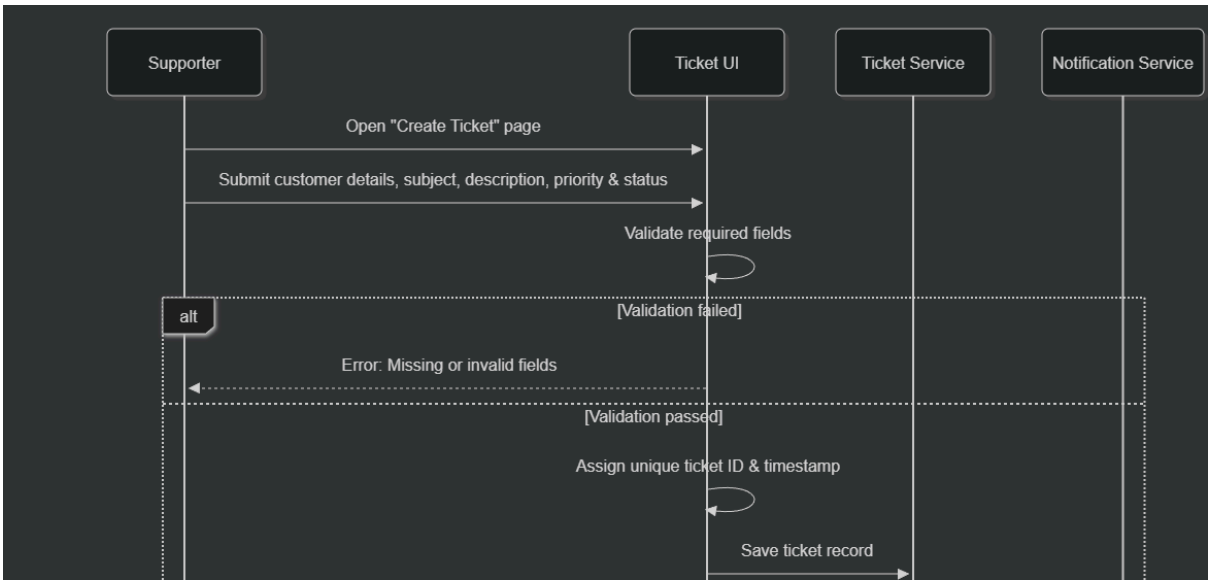


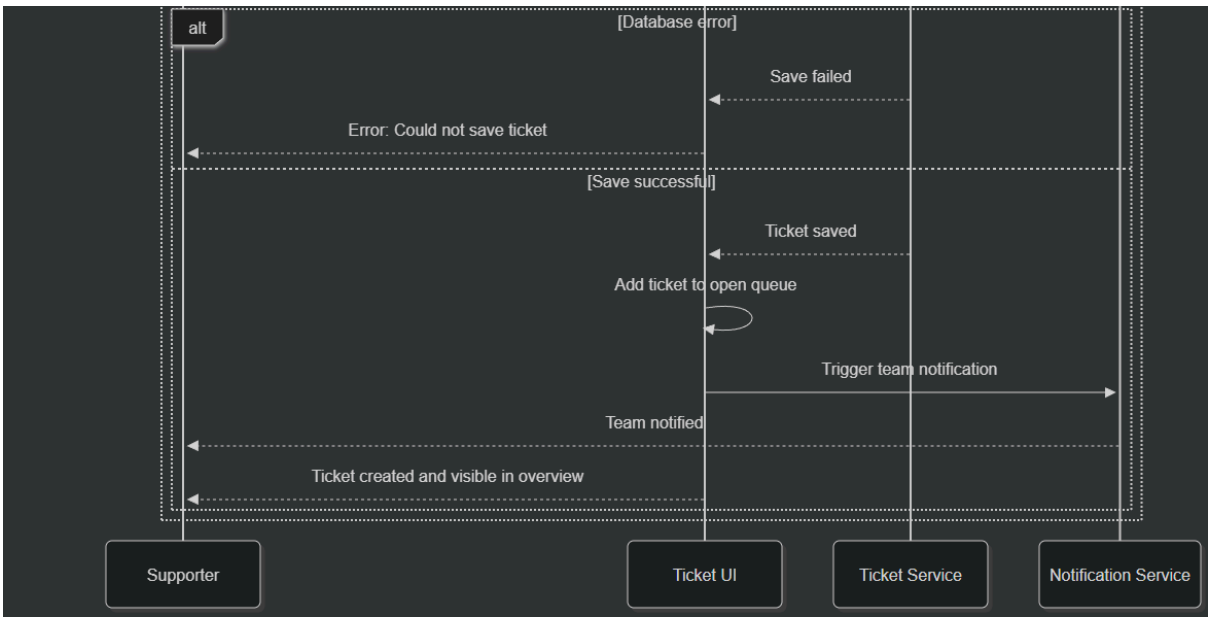


Usecase: 02 - Login
Aktør: Bruger
Forudsætninger: - Brugeren har oprettet en konto - Systemet er tilgængeligt via en browser
Succes scenarie: - Brugeren åbner login siden - Brugeren indtaster e-mail / brugernavn og adgangskode - Brugeren trykker “Login” - Systemet validere oplysningerne - Brugerens sendes ind til hovedsiden
Fejlkilder: - Brugeren indtaster forkert adgangskode - Kontoen er deaktiveret / spærret - systemet kan ikke forbinde til databasen


User
Auth Service
User Database
Email Service
Click "Forgot Password"
Submit email address
Validate email format
Look up user by email
alt
[User not found]
No record found
Error: User not found
[User found]
Return user record
Generate password reset token
Store token with expiry time
Request reset email with token link
Email sent with "Update Password" link
Click "Update Password" link
Submit new password
Validate token (check expiry)
alt
[Token expired]
Token invalid
Error: Link expired — request new one
[Token valid]
Update password hash
Invalidate reset token
Password updated
Redirected to login page
User
Auth Service
User Database
Email Service

Usecase: 03 - Forgot password
Aktør: Bruger
Forudsætninger: - Brugeren har en konto - Systemet er tilgængeligt via en browser - Brugeren har internetforbindelse
Succes scenarie: - Brugeren åbner login - Brugeren trykker “Forgot password” - Brugeren indtaster e-mail - Systemet sender en e-mail med “Update password” link - Brugerne trykker på “Update password” linket - Brugeren indtaster det nye password - Brugeren sendes til login
Fejlkilder: - Brugeren indtaster forkert e-mail - Systemet kan ikke forbinde til databasen - Brugeren eksisterer ikke





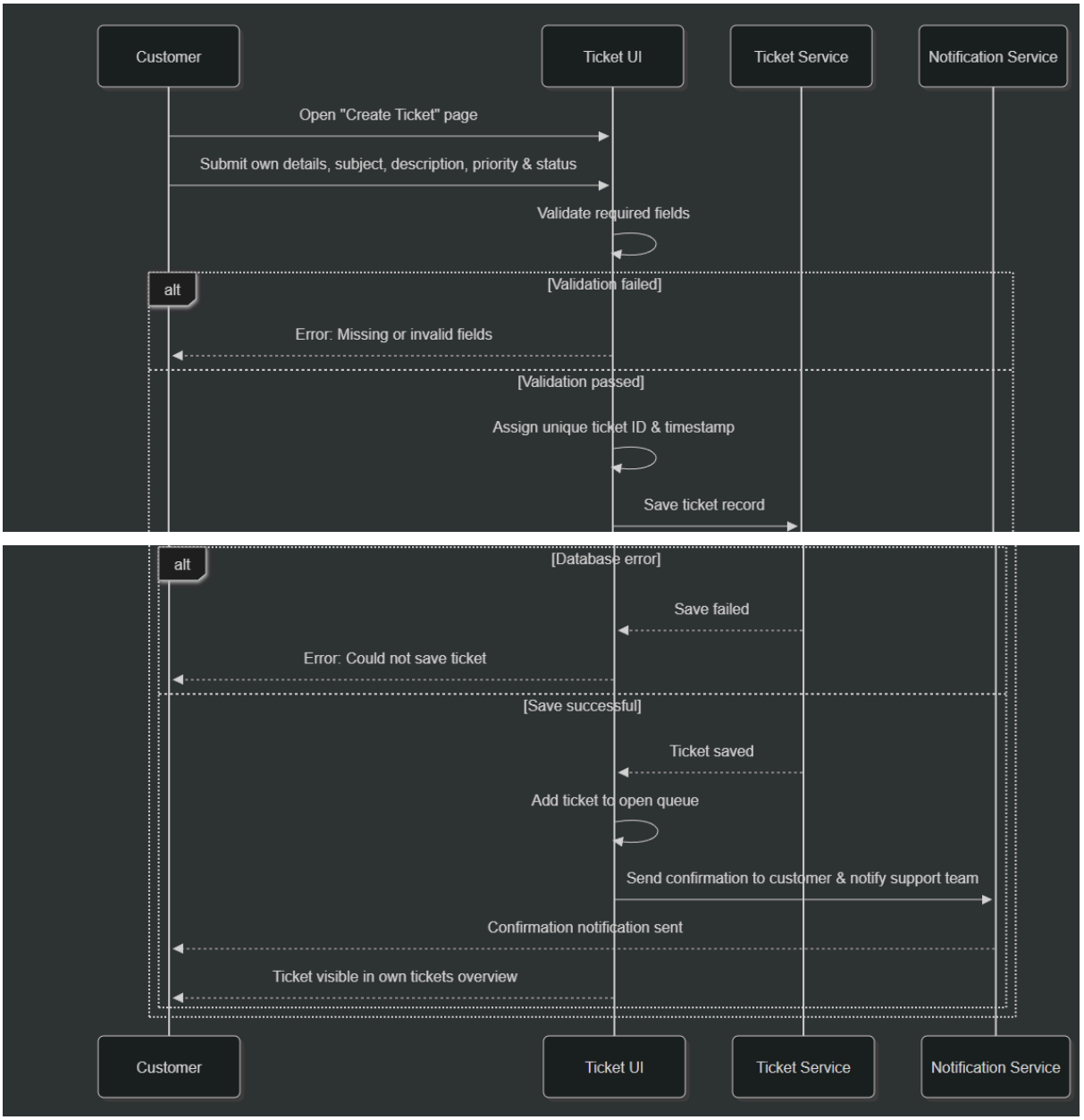
Usecase: 04 - Opret sag manuelt af supporter

Aktør: Supporter

Forudsætninger:
- Supporter er logget ind i systemet
- Supporter er tilknyttet en organisation (tenant)

Succes scenarie:
- Supporter åbner “Opret sag” siden
- Supporter indtaster kundens oplysninger (navn, e-mail)
- Supporter indtaster emne og beskrivelse
- Supporter vælger prioritet og status
- Supporter trykker “Opret sag”
- Systemet gemmer sagen i databasen
- Sagen vises i oversigten over åbne sager

Fejlkilder:
- Manglende obligatoriske felter
- Ugyldig e-mail adresse
- Systemet kan ikke forbinde til databasen



Usecase: 05 - Opret sag manuelt af Customer
Aktør: Customer
Forudsætninger: - Customerer logget ind i systemet - Customerer tilknyttet en organisation (tenant)
Succes scenarie: - Customer åbner “Opret sag” siden - Customer indtaster sine oplysninger (navn, e-mail) - Customer indtaster emne og beskrivelse - Customer vælger prioritet og status

- Customer trykker “Opret sag”
- Customer gemmer sagen i databasen
- Sagen vises i oversigten over åbne sager

Fejlkilder:
- Manglende obligatoriske felter
- Ugyldig e-mail adresse
- Systemet kan ikke forbinde til databasen

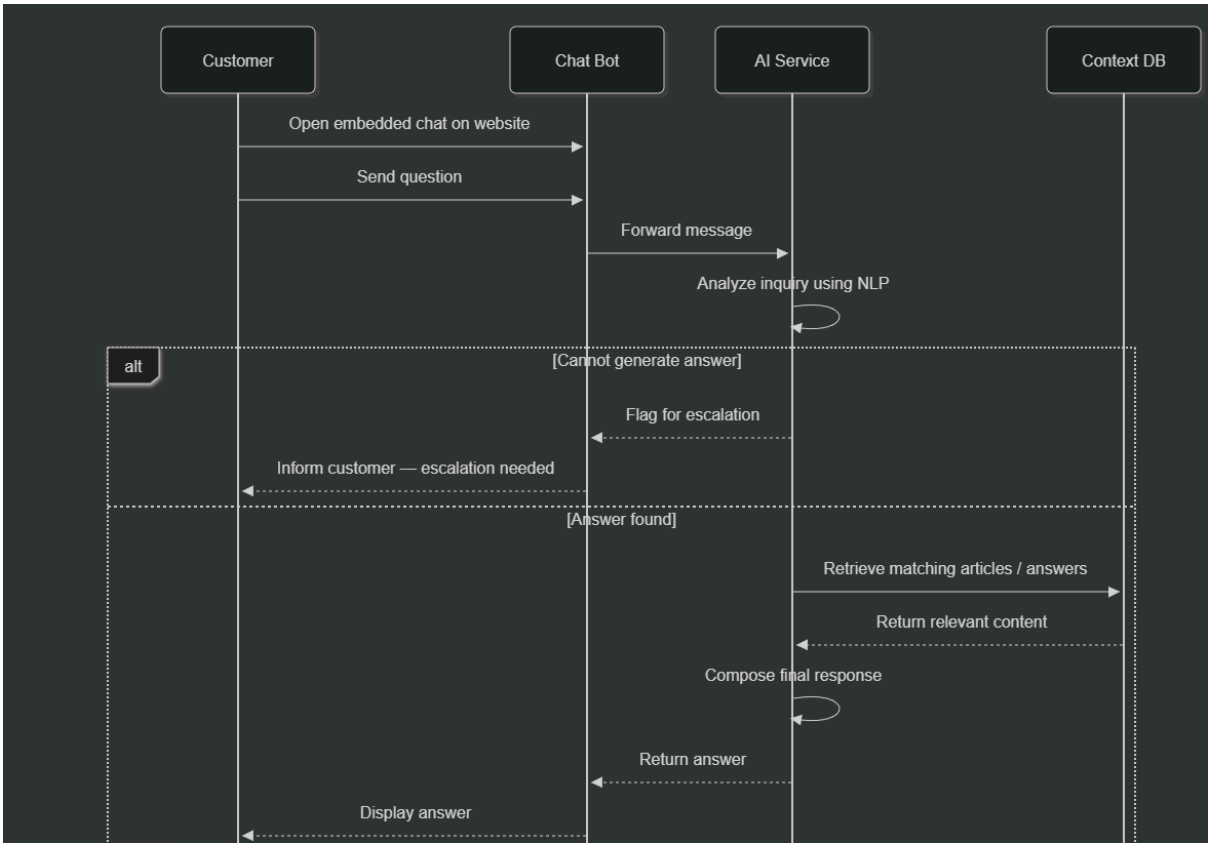
Usecase: 06 - Automatisk oprettelse af sag via e-mail

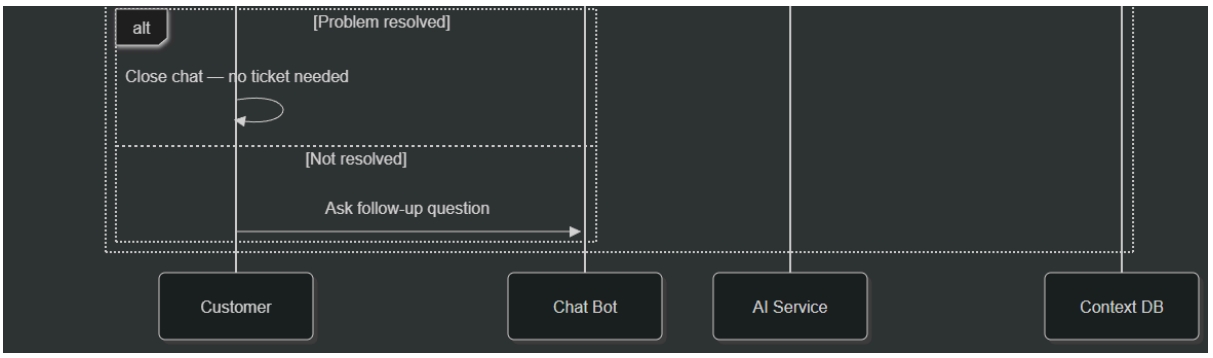
Aktør: Customer

Forudsætninger:
- Customer sender e-mail til virksomhedens support adresse
- E-mail service er aktiv
- Systemet er online

Succes scenarie:
- Kunden sender e-mail
- Systemet registrerer indgående e-mail
- Systemet opretter automatisk en ny sag i databasen
- Systemet knytter e-mail til sagen som første besked
- Kunden modtager en bekræftelsesmail
- Sagen vises i systemets sagsoverblik

Fejlkilder:
- E-mail service er nede
- Databasen er utilgængelig
- E-mail kan ikke parses korrekt





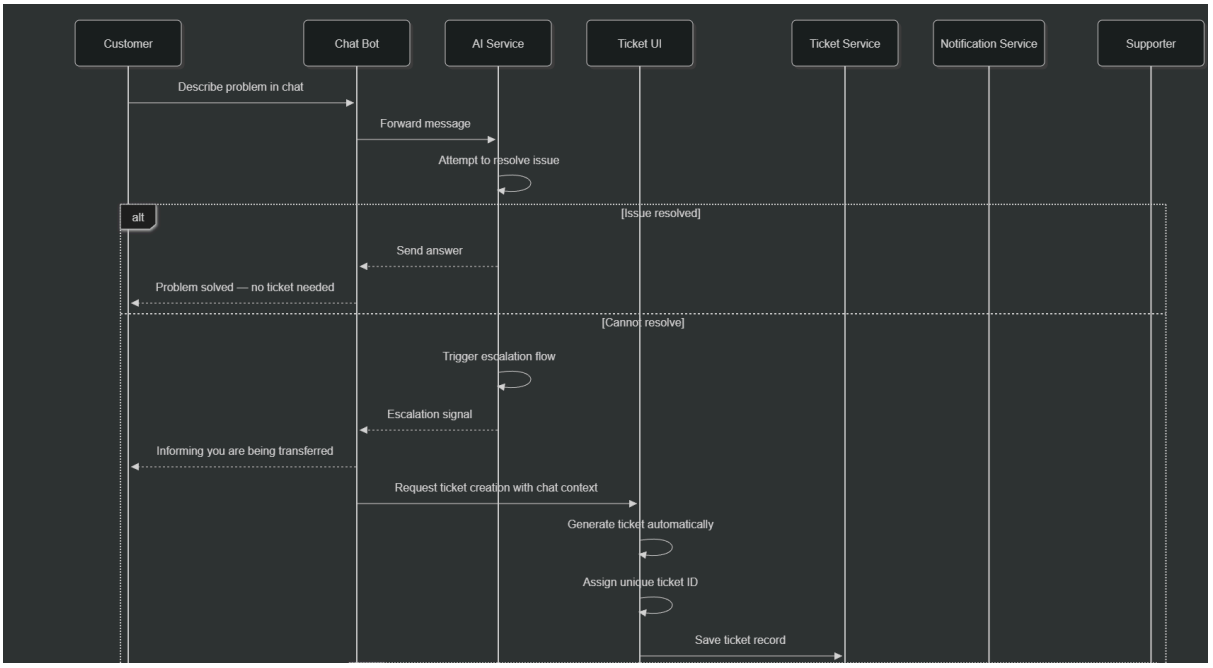
Usecase 07 - Chatbot håndterer henvendelse

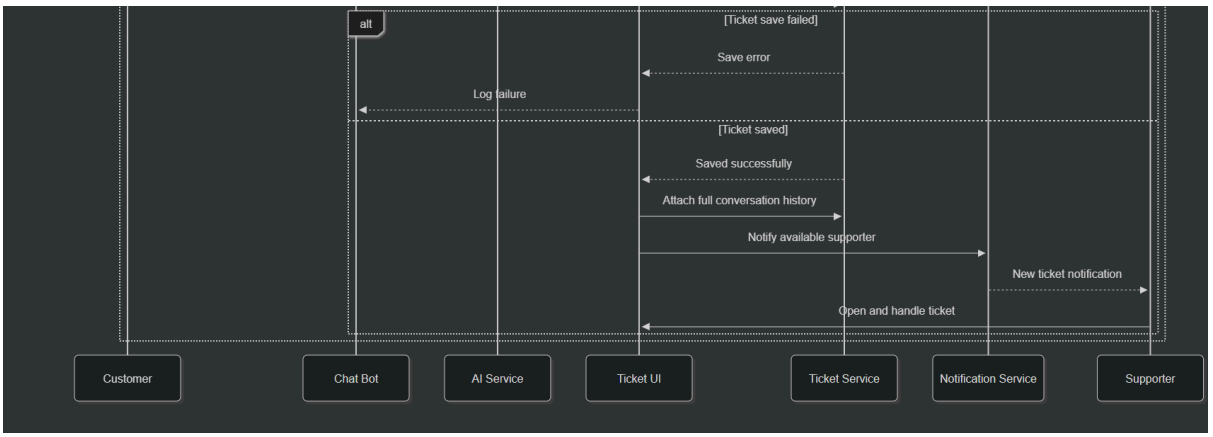
Aktør: Customer

Forudsætninger:
- Chatten er embedded på virksomhedens hjemmeside
- Chatbot service er aktiv

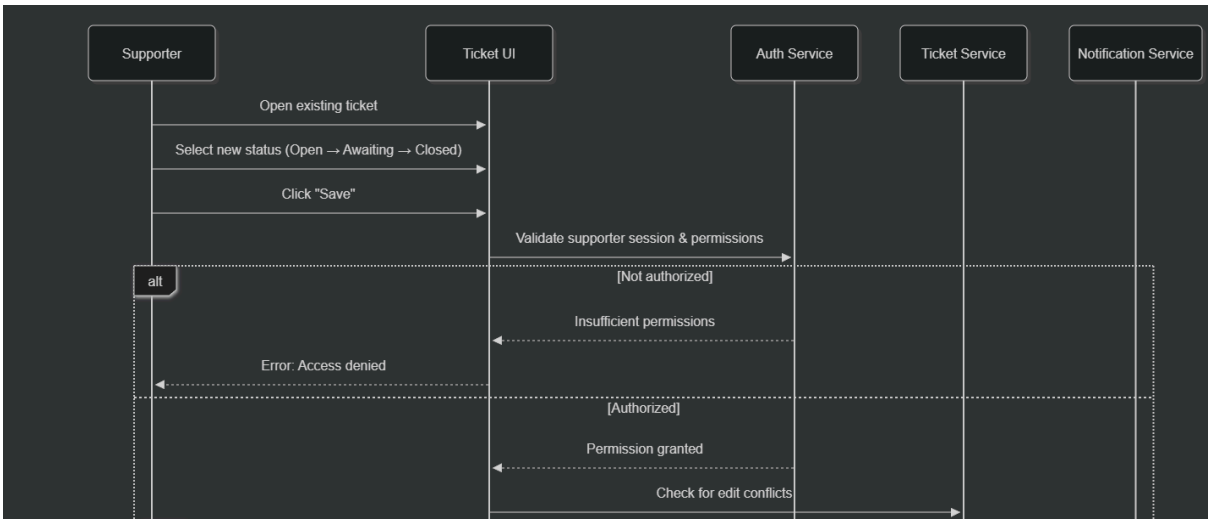
Succes scenarie:
- Kunden åbner chatten
- Kunden stiller et spørgsmål
- Chatbot analysere forespørgslen
- Chatbot giver svar på generelt spørgsmål
- Kunden får løst sit problem uden oprettelse af sag

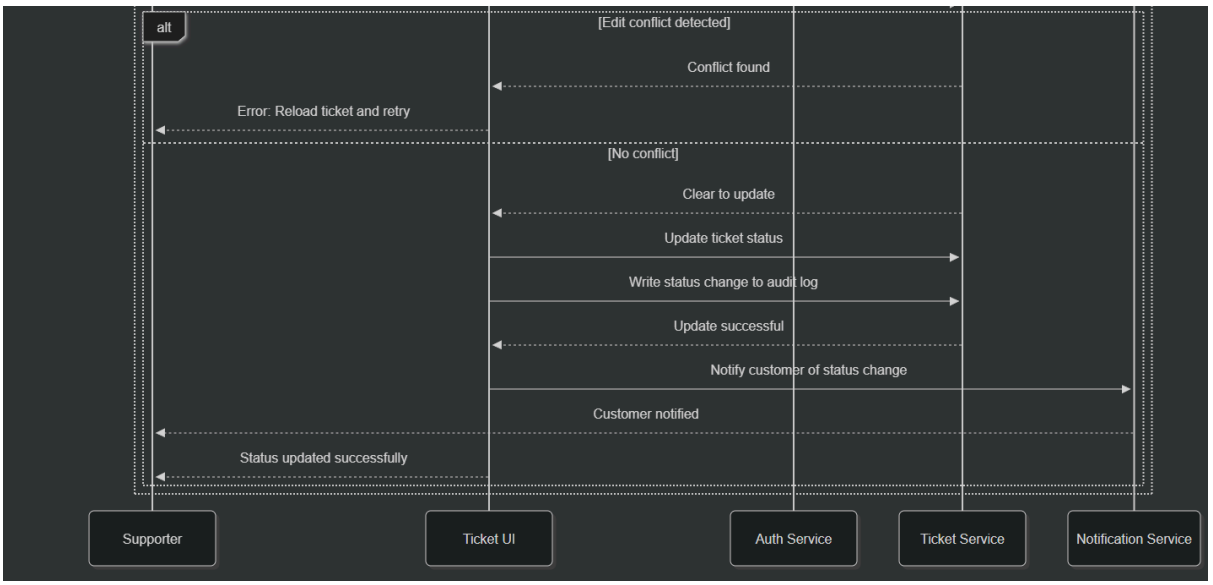
Fejlkilder:
- Chatbot service svarer ikke
- AI-model kan ikke genere svar
- Systemet kan ikke forbinde til backend





Usecase 08 – Eskalering fra chatbot til supporter
Aktør: Customer
Forudsætninger: - Chat session er aktiv - Chatbot vurderer at problemet kræver menneskelig support
Succes scenarie: - Kunden beskriver problem - Chatbot kan ikke løse problemet - Chatbot eskalerer samtalen - Systemet opretter automatisk en sag - Samtalehistorik vedhæftes sagen - Supporter får notifikation om ny sag½
Fejlkilder: - Sagen kan ikke oprettes - Samtalehistorik gemmes ikke korrekt - Notifikation sendes ikke





Usecase 09 – Ændre status på sag
Aktør: Supporter
Forudsætninger: - Supporter er logget ind - Der findes en eksisterende sag
Succes scenarie: - Supporter åbner sag - Supporter ændre status (fx Åben → Afventer svar → Lukket) - Supporter gemmer ændringen - Systemet opdatere status i databasen - Status Ændringen logges i historikken
Fejlkilder: - Manglende rettigheder - Databasen kan ikke opdateres - Konflikt ved samtidig redigering

4. Eksterne grænseflade-krav

4.1 Bruger-grænseflade

Brugergrænsefladen skal være webbaseret og tilgås via en browser uden nogle specifikke krav om installation. Designet skal være intuitivt og let at forstå for brugere uden teknisk viden, så en ny customer kan logge ind og begynde at kommunikere med virksomhedens support uden nogen oplæring/optræning. Applikationen skal præsentere et overskueligt layout med tydelig navigation mellem centrale områder,

	Admin	Manager	Support	Customer
Login	✅	✅	✅	✅
User Ticket Dashboard	✅ All tickets	✅ Own tickets and unassigned	✅ Own tickets and unassigned	❌
Ticket Tenant Dashboard	✅ All tickets	✅ All tickets	✅ Own tickets and unassigned	❌
Customer Ticket Dashboard	❌	❌	❌	✅

De forskellige Dashboards skal give de forskellige roller overblik over deres egne dashboard og de tickets som de har med at gøre. I de forskellige dashboards skal de forskellige tickets vises i kronologisk rækkefølge og vise et tidspunkt for hvornår den er blevet oprettet og anført. Chatbot-funktionen skal kunne integreres i Tenantens egen website så deres customers kan spørge om FAQs i forhold til tenantens virksomhed eller andet tilgængelig information.

4.2 Hardware-grænseflade

CCP App stiller ikke nogen krav til hardware, men applikationen er webbaseret og tilgås via en browser. Derfor er der krav til brugeren om at de skal have en enhed som har internetforbindelse.

4.3 Kommunikations-grænseflade

CCP App kommunikerer med dens tenants og deres customers gennem flere kanaler. Den primære måde foregår gennem tickets som sendes af customers i realtid mellem customers og support. Ved modtagelse af nyt ticket for support en push notifikation gennem applikationen. Derudover kan customers snakke med chatbot-tjenesten hvis tenants integrere den i deres website så kan de stille den FAQs og andet tilgængelig information.

4.4 Software-grænseflade

Systemet udvikles som en browserbaseret webapplikation, hvor brugerne tilgår løsningen via en almindelig webbrowser, mens de serverbaserede komponenter afvikles i et hostet miljø. På serversiden stilles der primært krav om, at styresystemet kan køre .NET-applikationer og understøtte containerbaseret orkestrering. Løsningen kan derfor afvikles på både Windows- og Linux-baserede servere, men er designet med fokus på containerbaseret deployment, hvor Linux typisk er det foretrukne standardmiljø.

Applikationen er opbygget af flere softwarekomponenter, som kommunikerer gennem klart definerede grænseflader. Webapplikationen (Razor Pages) udgør præsentationslaget og anvender HTTP-baserede API-kald til at kommunikere med backend, som implementeres i ASP.NET Web API. Backend udstiller endpoints, der håndterer kerne scenarier som brugeradministration, oprettelse af tickets, håndtering af kanal adgang, afsendelse af beskeder og hentning af kommunikationshistorik for en given sag.

Datagrundlaget lagres i PostgreSQL, og database adgangen håndteres via Entity Framework, der anvendes som ORM til objekt-relation mapping, migrations og generelle database operationer.

Afsendelse af e-mailnotifikationer håndteres af en separat e-mailservice, som stiller et internt service-interface eller API til rådighed. Backend kan kalde denne service, når der registreres nye beskeder i en chat, og e-mail servicen fungerer dermed som integrationslag mod en ekstern mailserver eller mailudbyder.

Da systemet er serviceorienteret og orkestreres med .NET Aspire, afvikles de enkelte services som selvstændige processer eller containere. Kommunikation og afhængigheder håndteres gennem konfiguration og service discovery, hvilket gør det muligt at køre systemet både lokalt under udvikling og i et produktionsmiljø uden at ændre på den overordnede softwaregrænseflade.

5. Krav til programmellets ydelse

Da CPP App er et realtidssystem til customer communication, har det stor betydning, hvor hurtigt en henvendelse sendes og modtages. Det er derfor vigtigt, at svartiden er høj, så support og customer ikke venter mere end maksimalt 5 sekunder på at sende eller modtage en besked i en ticket.

Det er derfor vigtigt at programmet testes for hvordan det reagerer under load og ved større datamængder som hvis der er bliver sendt mere end 100.000 tickets så skal den håndtere det og så frem det fejler skal den fejle yndefuldt uden at crashe browser.

For at opnå den forventede ydelse er det derfor nødvendigt at designe programmet så der ikke bliver sendt unødvendigt data for at holde mængden af data nede på en minimum. Derudover er det vigtigt at optimere systemets ressourceforbrug, så det er så optimalt som muligt og at der altid er ledige ressourcer til nye forespørgsler og svartiden derfor er hurtigere.

6. Kvalitetsfaktorer

6.1 Pålidelighed

For at sikre vores systems stabilitet og pålidelighed indsamler vi løbende logs samt performance-data. Denne data anvendes til at overvåge systemet og opdage eventuelle brute force-angreb mod brugernes konti eller andre performance issues.

6.2 Vedligeholdelsesvenlighed

For at sikre kodens kvalitet anvender vi code metrics, som giver os et overblik over kodens vedligeholdelsesniveau. Vi sigter efter at vores kode opnår en Maintainability Index i den grønne kategori, det som svarer til en score mellem 20-100. Derudover går vi efter at holde vores Cyclomatic Complexity score så lav som muligt. Denne score måler kodens kompleksitet, og jo højere score er, jo sværere bliver koden at vedligeholde.

6.3 Udvidelse Venlighed

Det er designet så systemet er skalerbart og nemt kan udvides med nye funktioner.

6.4 Brugervenlighed

Vores UI kommer til at være meget intuitiv og simpelt ved hjælp af både funktionernes navne og ikoner, så brugeren nemmest og hurtigst muligt kan lære at finde rundt i vores webapplikation, uden at de enten ender på den forkerte ticket eller overhovedet ikke finde rundt på webapplikationen, fordi der er for meget tekst som ikke er nødvendigt.

6.5 Genbrugbarhed

Det er vigtigt at vi laver systemet, så det er genbrugeligt, da det hjælper med at bringe mængden af kode som skal vedligeholdes ned. Det er dog vigtigt at

man tænker over om det er relevant at designe det som genbrugeligt hvis ikke så ender man med mere kompleksitet end ved at duplikere koden.

Vores system er derfor designet så at hvis at kodens “Cyclomatic Complexity” har en værdi højere end 50 skal koden refactors for nedbringe dens kompleksitet for at holde en god standard.

6.6 Effektivitet

Systemet er designet med en microservice arkitektur som gør det muligt at opskalere individuelle services an på behovet. Det er derfor også vigtigt at hver service er så effektiv som muligt så servicerne ikke bliver en flaskehals for systemet. Det vil sige, at de skal effektiviseres, så når de query databasen, skal det ske asynchronously for ikke at låse ressourcerne mens de afventer svar fra databasen. Derudover vil vi bruge Database connection pooling til at sørge for at servicere har mere end en database connection samtidig gør det også at der ikke skal åbnes nye connection ved hver query men at man er i stand til at genbruge allerede eksisterende forbindelser.

Systemet skal også gøres så memory effektivt så vi bruger så lidt som muligt og så garbage collector køre så lidt som muligt og så effektivt som muligt når den gør.

7. Andre krav

7.1 UI design

UI-designet i CCP App skal have fokus på enkelhed, overblik og en intuitiv brugeroplevelse, hvor hurtig og effektiv kommunikation er i centrum. Brugergrænsefladen skal udformes på en måde, så både nye og erfarne brugere let kan finde rundt i systemet uden unødig forvirring. Designet skal derfor fremstå minimalistisk og funktionelt, hvor de mest centrale handlinger, såsom at åbne en chat, sende beskeder og bruge chat-botten, altid er nemme at finde og benytte.

Det visuelle design skal være konsekvent på tværs af hele systemet gennem brug af ensartede farver, skrifttyper og komponenter. Knapper, inputfelter og øvrige interaktive elementer skal fremstå tydelige og lette at forstå, så brugeren hurtigt kan aflæse deres funktion. Der skal desuden lægges vægt på høj læsbarhed, hvilket blandt andet indebærer passende kontrast, tydelige skriftstørrelser og god afstand mellem elementerne, så indholdet præsenteres på en behagelig og overskuelig måde. Brugergrænsefladen udvikles primært med fokus på brug på computer, men designes responsivt, så den også i et vist omfang kan anvendes på tablet og mobiltelefon

Applikationens visuelle opbygning skal tage udgangspunkt i et tydeligt og genkendeligt layout. Der skal være en klar opdeling mellem ticket dashboardet og den aktive ticket, så medarbejderen hurtigt kan skifte mellem forskellige kundesager og samtidig bevare overblikket over den igangværende kommunikation. I ticket dashboardet skal sagens emne og kundenavn være synlige, og det skal tydeligt fremgå, når der er ny aktivitet eller ulæste beskeder, så medarbejderen hurtigt kan identificere, hvor der kræves opmærksomhed.

Den aktive ticket skal fungere som det primære fokusområde og vise beskeder i kronologisk rækkefølge, hvor der visuelt skelnes tydeligt mellem medarbejderens egne svar, kundens henvendelser og eventuelle chatbot-svar, så dialogen altid er let at følge og overskue.

7.2 Sikkerhed

Sikkerhed er et centralt krav i CCP systemet, da platformen håndterer personfølsomme oplysninger om kunder og interne kommunikationsforløb. Sikkerheden i systemet bygger på flere lag, der tilsammen beskytter data, adgang og kommunikation.

Al adgang til systemet kræver login og håndteres gennem Keycloak, som fungerer som systemets centrale authentication provider. Keycloak udstedet JWT-tokens (JSON Web Tokens), der anvendes til at verificere brugerens identitet ved hvert API-kald. Tokens har en begrænset levetid og fornyes automatisk via refresh tokens, så en kompromitteret session ikke forbliver gyldig i lang tid.

Adgangsstyringen er rollebaseret (RBAC - Role-Based Access Control) og organiseret i fire roller: Admin, Manager, Supporter og Customer. Rollerne er defineret i Keycloak og knyttes til brugerne via grupper under den pågældende tenants organisation. Dette sikrer, at en bruger kun kan tilgå de funktioner og data, som er relevante for deres rolle, og at rettigheder administreres et centralt sted frem for spredt i applikationen.

Al kommunikation mellem klient og server foregår over HTTPS, som krypterer data under transport og forhindrer aflytning og man-in-the-middle-angreb. Dette gælder både for brugergrænsefladen og for API-kald mellem de interne services. Adgangskoder gemmes aldrig i klartekst i database, men håndteres udelukkende af Keycloak, som anvender sikker hashing.

Systemet skal beskyttes mod de mest udbredte angrebstyper, så for at forhindre SQL-injection anvender vi Entity Framework som ORM, der parameteriserer alle databaseforespørgsler automatisk og dermed eliminerer risikoen for direkte SQL-manipulation. For at beskytte mod brute force-angreb

på login-formularen registrerer systemet gentagne fejlslagne loginforsøg via logs og Keycloak konfigureres til at spærre konti midlertidigt efter et defineret antal fejlforsøg. (Note: Tænker at dette er noget der er muligt at definere i Keycloak men da jeg ikke er sikker har jeg lige markeret det med gult for nu)

7.3 CI/CD

For at sikre en stabil og effektiv udviklingsproces anvendes CI/CD i projektet. CI står for " Continuous Integration" og handler om, at ny kode løbende bliver samlet, testet og valideret automatisk, når der bliver lavet ændringer i projektet. CD står for "Continuous Delivery eller Continuous Deployment" og handler om, at systemet efterfølgende kan gøres klar til levering eller automatisk udrulles til et miljø

Formålet med at bruge CI/CD er at reducere fejl, gøre udviklingsprocessen mere ensartet og sikre, at nye funktioner kan integreres uden at ødelægge eksisterende funktionalitet. Da CCP App består af flere services, er det vigtigt at ændringer i den ene del af systemet hurtigt kan testes op imod resten af løsningen.

Fordele ved CI/CD

Brugen af CI/CD giver flere fordele i projektet:

- Automatisk validering af kode ved ændringer
- Hurtigere fejlfinding, fordi problemer opdages tidligt
- Mere stabil og ensartet build-proces
- Mindre risiko for at ny kode bryder eksisterende funktioner
- Hurtigere levering af nye funktioner og fejlrettelser
- Bedre samarbejde mellem udviklere, fordi alle arbejder mod samme pipeline
- Mulighed for ensartet deployment til test- og produktionsmiljøer

For et system som CCP App er dette særligt relevant, fordi platformen skal være driftsikker og kunne videreudvikles over tid uden at miste kvaliteten.

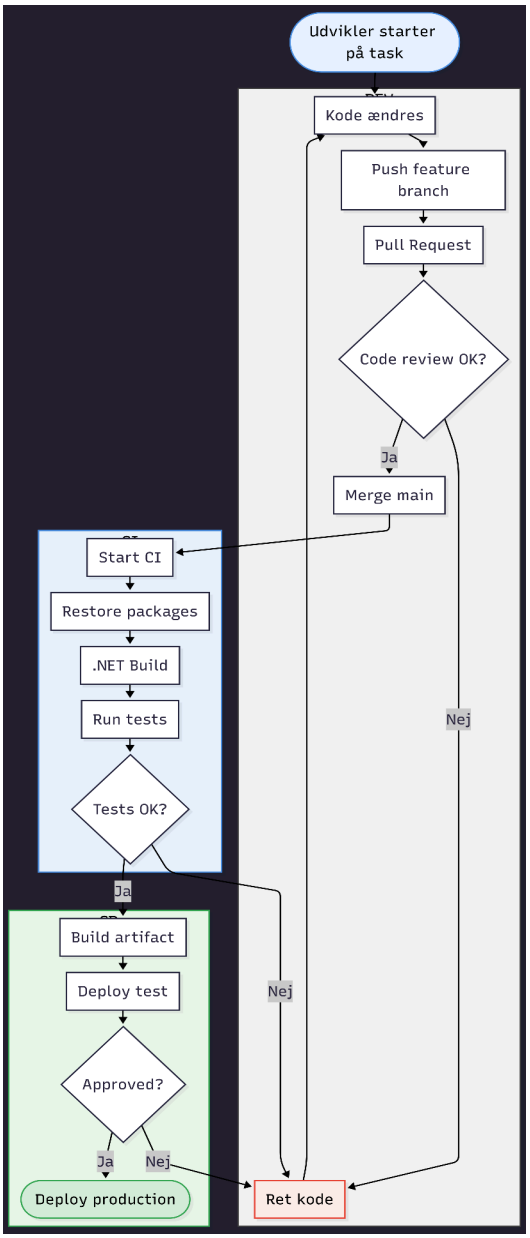
CI/CD i dette projekt

I projektet kan CI/CD-pipelinen eksempelvis bestå af følgende trin:

1. En udvikler laver ændringer i koden og pusher dem til repository.
2. Pipelinens build-trin starter automatisk.
3. Koden kompileres, og afhængigheder gendannes.
4. Automatiske tests køres, fx unit tests og integrationstests.
5. Kvalitetskontrol udføres, fx linting, kodeanalyse eller måling af code metrics.
6. Hvis alle trin lykkes, bygges applikationen til et deploy bart artifact eller container image.

7. Løsningen bliver deployed til et testmiljø.
8. Efter godkendelse kan løsningen blive deployed til produktionsmiljøet.

   På den måde sikres det, at kun kode, der er bygbar og testet, bliver gjort klar til drift



CI/CD pipeline for CCP-platformen.
Diagrammet viser udviklingsprocessen fra kodeændringer og Pull Request til Continuous Integration med build og tests, hvorefter systemet deployes gennem Continuous Delivery til test- og produktionsmiljø

7.4 Personlige SWOT analyser

Oliver:

Styrker:

- Jeg er en full-stack udvikler. Jeg kan arbejde med både frontend og backend, hvilket gør mig fleksibel og nyttig på tværs af forskellige dele af et projekt.
- Stærk frontend-implementering. Bliver jeg givet et design, kan jeg genskabe det i kode med høj præcision så længe det er teknisk muligt.
- Jeg kender mig selv godt. Jeg har et ret ærligt billede af, hvor jeg er stærk, og hvor jeg kæmper, hvilket hjælper mig med at planlægge og kommunikere bedre i et team.
- Solid backend-forståelse. Nye backend-teknologier tager mig lidt længere tid at sætte mig ind i, men jeg finder altid ud af det.

Svagheder:

- Nye teknologier tager tid. Når jeg støder på et nyt værktøj, framework eller sprog, har jeg brug for en indkøringsperiode, før jeg er fuldt produktiv, og jeg har ind imellem brug for hjælp undervejs.
- Jeg falder tilbage på ren CSS. Jeg bruger ikke tit CSS-frameworks som Tailwind eller lign. så meget som jeg nok burde, hvilket kan gøre mig langsommere sammenlignet med en mere struktureret arbejdsgang.
- Jeg kan ikke designe. Jeg er god til at bygge grænseflader ud fra et eksisterende design, men hvis du beder mig om at finde på noget fra bunden, er det slet ikke min stærke side.

Muligheder:

- Mine igangværende skole og personlige projekter. Projekter som CCP eller Arcane Archive giver mig et reelt miljø til at afprøve nye teknologier uden så meget pres som i en professionel sammenhæng.
- Bevidst læring af nye frameworks. Da jeg ved, at indkøring er noget, jeg kæmper med, kan jeg vende det om ved bevidst at tage et nyt værktøj op per projekt og give mig selv tid til at lære det ordentligt.

Trusler:

- Deadlines. Den tid, jeg har brug for til at blive fortrolig med nye ting, kan kollidere med stramme deadlines, hvilket skaber stress og tvinger mig nogle gange til at tage genveje, jeg ikke er tilfreds med.
- Tech landskabet bevæger sig hurtigt. Der er altid noget nyt at lære, og da jeg allerede er langsom til at tage nye ting op, er jeg nødt til at arbejde aktivt for ikke at falde bagud.
- Imposter syndrom. Jeg undervurderer nogle gange mine egne evner, især på frontend-siden. Det kan forhindre mig i at sige noget eller tage udfordringer op, som jeg faktisk er i stand til at håndtere.

Carl-Emil:
Styrker:

- Jeg ser mig selv som en kompetent full-stack udvikle**r** med erfaring inden for både backend- og frontend-udvikling. Jeg finder begge områder lige interessante at arbejde med, men oplever backend som en smule mere komplekst end frontend. Mine kompetencer indenfor begge områder er dog relativt balancerede, hvilket giver mig et godt udgangspunkt i projekter, der arbejder på tværs af teknologier..
- Jeg har en god evne til selvrefleksion, og jeg er ærlig omkring mit faglige niveau. Det gør mig i stand til realistisk at vurdere mine kompetencer og planlægge mit arbejde derefter, så jeg kan løse opgaver effektivt og målrettet.
- Jeg er motiveret for at lære nye teknologier og værktøjer. Jeg er samtidig bevidst om, at nye teknologier kræver tid at blive fortrolig med, og jeg prioriterer derfor at opnå en solid forståelse.
- Jeg ser mig selv som socialt velfungerende og samarbejdsorienteret, hvilket gør mig i stand til at indgå konstruktivt i gruppearbejde. Jeg trives godt i teams og bidrager positivt til samarbejdet i større projektgrupper.

Svagheder:

- Når jeg arbejder med nye teknologier, værktøjer eller frameworks, har jeg ofte brug for tid til at lære det, før jeg kan arbejde fuldt effektivt. I begyndelsen kan jeg derfor have behov for sparring eller hjælp, indtil jeg opnår en solid forståelse af teknologien
- Som følge af en tidligere **hjernerystelse** oplever jeg, at jeg ikke kan arbejde i meget lange perioder uden pauser. Det kan også betyde, at jeg nogle gange skal bruge ekstra tid på at sætte mig ind i eksisterende kode og fastholde overblik over større kodebaser.
- jeg ser backend-udvikling som en smule mere kompleks end frontend. Det kan gøre, at jeg i nogle situationer bruger længere tid på at forstå og implementere backend-relaterede løsninger. Samtidig er backend et område, jeg finder meget interessant, og derfor arbejder jeg aktivt på at udvikle mine kompetencer inden for dette område.

Muligheder:

- personlige projekter, såsom CCP, Movie Shaker App og Trænings Tracker App, giver mig mulighed for at afprøve nye teknologier i et relativt trygt miljø. Det giver mig mulighed for at eksperimentere og udvikle mine kompetencer uden det samme pres, som kan være i en professionel sammenhæng.

- Jeg kan arbejde mere bevidst med at lære nye frameworks og værktøjer. Da jeg ved, at nye teknologier kan tage tid at lære, kan jeg aktivt vælge at fokusere på ét nyt værktøj pr. projekt og få tid til at opnå en solid forståelse af det.

Trusler:

- Mine følger af **hjernerystelse** kan påvirke min koncentration og gøre det sværere for mig at tilegne mig ny viden over længere perioder. Det kan i nogle situationer gøre læringsprocesser langsommere end ønsket.
- Stramme deadlines kan være en udfordring. Den tid, jeg ofte har brug for til at blive fortrolig med nye teknologier eller værktøjer, kan kollidere med projektets tidsrammer, hvilket kan skabe pres og i enkelte tilfælde føre til løsninger, jeg ikke er fuldt tilfreds med.
- imposter-syndrom, her undervurderer jeg mine egne kompetencer, særligt inden for backend. Det kan i nogle situationer gøre mig mere tilbageholdende med at tage initiativ eller påtage mig opgaver, som jeg i praksis godt ville kunne løse.

Emil:

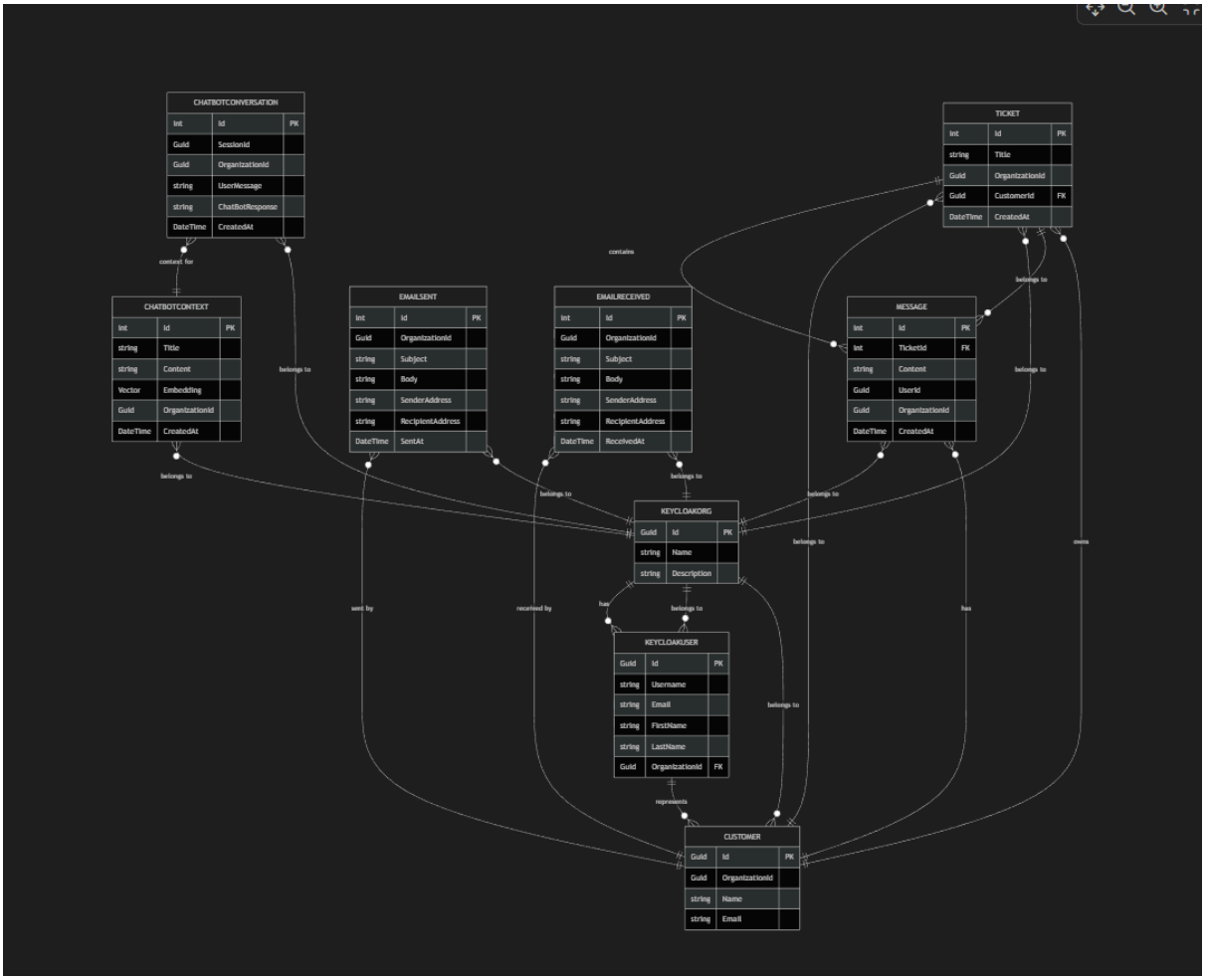
Aksel:

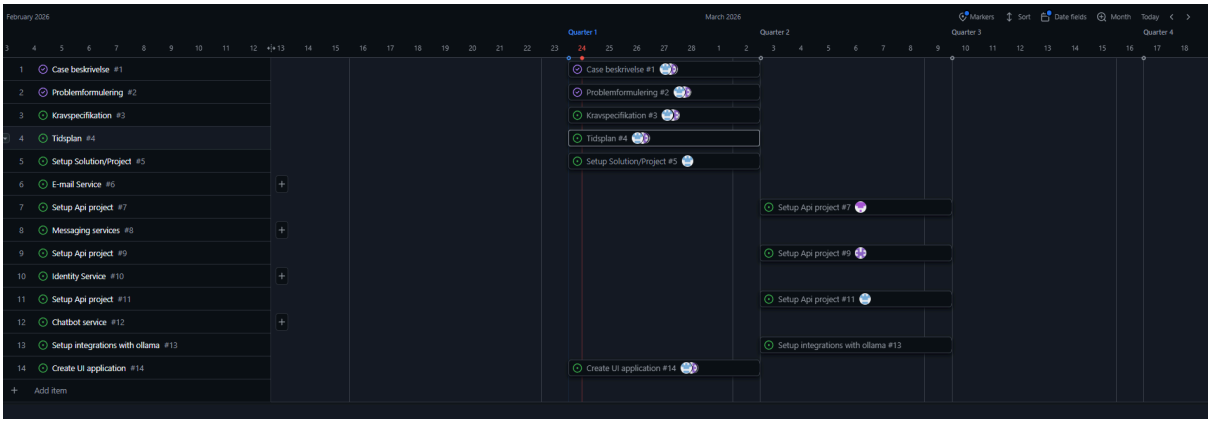
8. Skærmbilleder

Vi har designet nogle wireframes/proof of concepts til hvordan vores webapplikation kan ende med at se ud med fokus på brugervenlighed så de nemt og intuitivt kan finde rundt.

9. E/R diagram

Work in progress



10. Estimeret tidsplan

Eksempel på vores tidsplan